Reporte de Entrega: Implementación del Patrón Decorator

Curso / Materia: Diseño de Software

Tema: Patrón de Diseño Decorador (Decorator Pattern)

Isaac Alejandro Garcia Ruiz

739528

Allison Briseyda Ávila Moctezuma

757985

1. Estructura de Archivos del Proyecto

El proyecto sigue la convención Orientada a Objetos de mantener **una sola clase principal por archivo de código fuente**:

Decorator-Pattern-Message-Decorators/

```
├── src/
│   ├── message.py
│   ├── plain_message.py
│   ├── message_decorator.py
│   ├── urgent_decorator.py
│   └── timestamp_decorator.py
├── tests/
│   └── test_decorators.py
└── conftest.py
```

2. Código Fuente de la Solución

Decorador Base Abstracto: src/message_decorator.py

Clase base para todos los decoradores que implementa la interfaz Message y almacena la referencia del mensaje envuelto (wrapped).

```python
from .message import Message


class MessageDecorator(Message):
    def __init__(self, wrapped: Message):
        self.wrapped = wrapped

    def render(self) -> str:
        return self.wrapped.render()
```

Decorador Concreto Urgente: src/urgent_decorator.py

Agrega el prefijo URGENT: al mensaje renderizado.

```python
from .message_decorator import MessageDecorator


class UrgentDecorator(MessageDecorator):
    def render(self) -> str:
        return f"URGENT: {self.wrapped.render()}"
```

Decorador Concreto Marca de Tiempo: src/timestamp_decorator.py

Agrega un timestamp entre corchetes al inicio del mensaje renderizado.

```python
from .message_decorator import MessageDecorator


class TimestampDecorator(MessageDecorator):
    def __init__(self, wrapped, timestamp: str = "2026-10-30"):
        super().__init__(wrapped)
        self.timestamp = timestamp

    def render(self) -> str:
        return f"[{self.timestamp}] {self.wrapped.render()}"
```

3. Pruebas Unitarias

Archivo: tests/test_decorators.py

Pruebas enfocadas que comprueban el funcionamiento individual, el apilamiento de decoradores y la relevancia del orden en el que se envuelven.

```python
from src.messages.plain_message import PlainMessage
from src.messages.urgent_decorator import UrgentDecorator
from src.messages.timestamp_decorator import TimestampDecorator


def test_individual_and_stacked_decorators():
    base_msg = PlainMessage("Server connection lost")
    timestamp = "2026-10-30 14:00"

    assert base_msg.render() == "Server connection lost"

    urgent_msg = UrgentDecorator(base_msg)
    assert urgent_msg.render() == "URGENT: Server connection lost"

    stacked_msg = TimestampDecorator(
        UrgentDecorator(base_msg),
        timestamp=timestamp
    )
    assert stacked_msg.render() == f"[{timestamp}] URGENT: Server connection lost"

    reverse_stacked_msg = UrgentDecorator(
        TimestampDecorator(base_msg, timestamp=timestamp)
    )
    assert reverse_stacked_msg.render() == f"URGENT: [{timestamp}] Server connection lost"
```

TEST

```
(.venv) PS C:\Users\barri\OneDrive\Desktop\escuela\5to semestre\diseno de software\Decorator-Pattern-Message-Decorators> python -m pytest
tests\test_plain_message.py .                                                                                                    [100%]

============================================================ 2 passed in 0.09s ============================================================
```